

DESENVOLVIMENTO MOBILE

AULA 5 – SENSORES DO DISPOSITIVO

GPS · acelerômetro · giroscópio · câmera · CESAR School 2026.2



ATÉ ONTEM, SEU APP SÓ SABIA O QUE VOCÊ DIGITOU NELE.

HOJE ELE DESCOBRE ONDE ESTÁ, COMO ESTÁ SENDO SEGURADO E O QUE ESTÁ VENDENDO.

O SISTEMA OPERACIONAL NÃO ENTREGA DE GRAÇA

- 1 Ele **pergunta ao dono do aparelho** antes → permissão
- 2 Ele entrega **quando conseguir** → é assíncrono, e pode falhar
- 3 Ele **cobra bateria** enquanto você lê

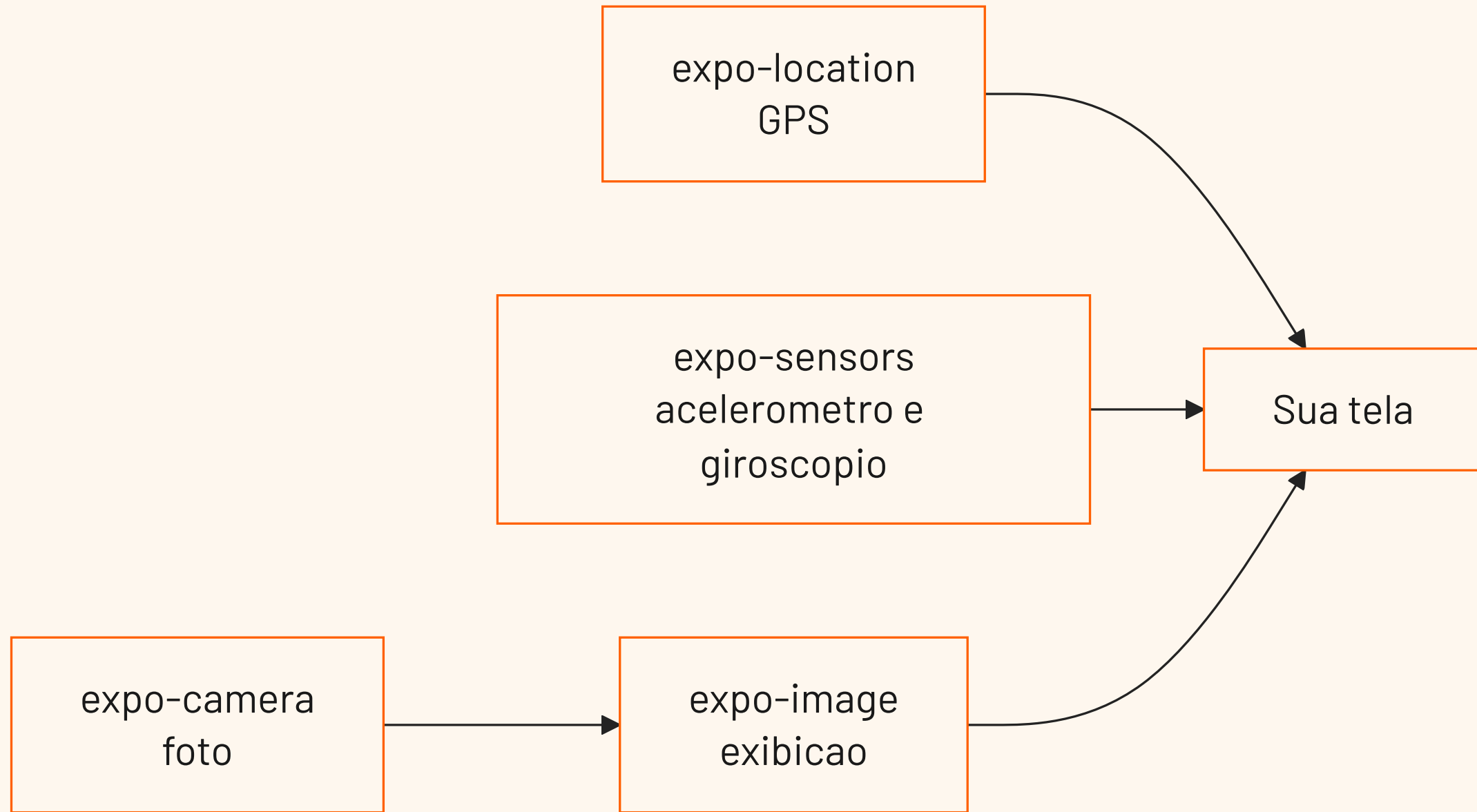
NÃO EXISTE `const local = pegarLocalizacao()`.

AGENDA – 3 HORAS

Bloco	Tempo
Abertura + a virada do dia	8 min
1. O contrato de três tempos	17 min
2. GPS com expo-location	30 min
☕ Intervalo	10 min
3. Checkpoint – permissão e GPS	7 min
4. Acelerômetro e giroscópio	25 min
5. Câmera com expo-camera	28 min
6. expo-image	15 min
7. Checkpoint – sensores e imagem	7 min
8. Hands-on guiado	25 min
9. Atividade aplicada + fechamento	8 min



QUATRO PACOTES, UM CONTRATO SÓ



OS TRÊS PRIMEIROS PRODUZEM DADO. O QUARTO MOSTRA.

AO FINAL DA AULA VOCÊ DEVE CONSEGUIR

- 1 Descrever o contrato **permissão** → **leitura** → **parada**
- 2 Tratar `granted`, `canAskAgain: false` e "serviço desligado" **de formas diferentes**
- 3 Escolher entre **leitura pontual** e **assinatura contínua**
- 4 Obter a posição com a `Accuracy` certa e ler `coords` corretamente
- 5 Ler acelerômetro e giroscópio sabendo **o que** cada um mede e **em que unidade**
- 6 Montar a tela de câmera com o gate de permissão de **três** estados
- 7 Exibir a imagem com `contentFit`, `placeholder` e `transition`



PARTE 1

O CONTRATO DE TRÊS TEMPOS



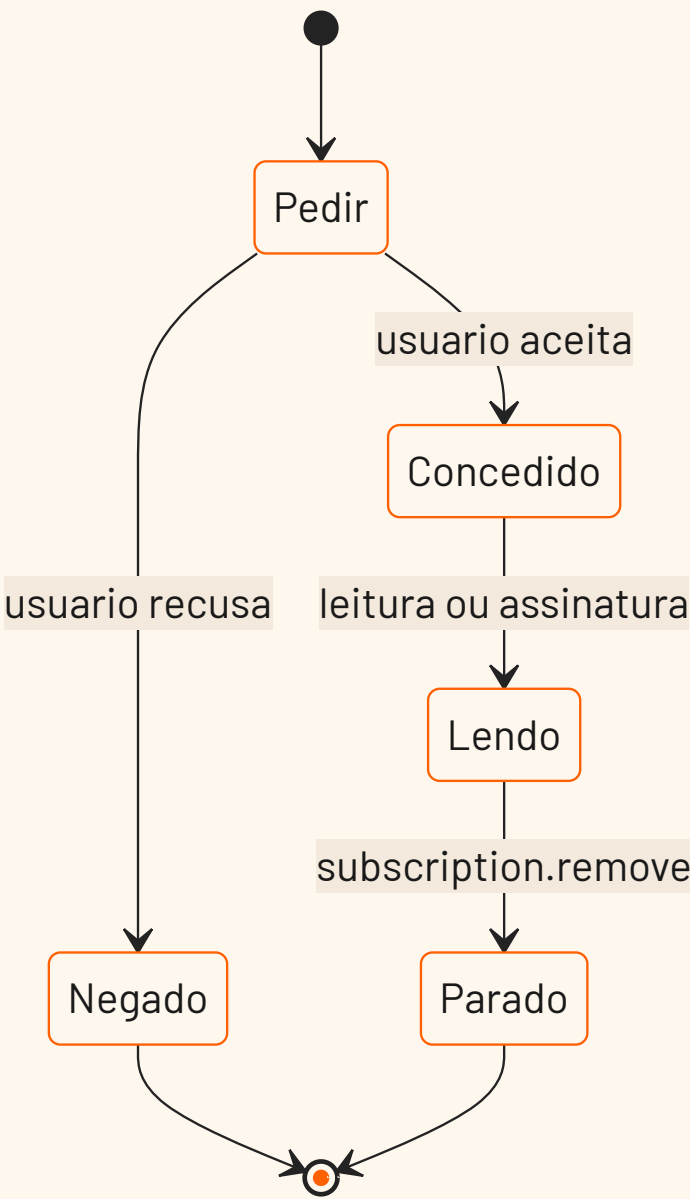
UM SENSOR NÃO É UMA VARIÁVEL

É...	Porque
compartilhado	outros apps querem o mesmo hardware
protegido	o SO exige consentimento do dono
caro	cada leitura custa bateria

TODA API DE SENSOR É **async** E PODE DIZER NÃO.



OS TRÊS TEMPOS



O TEMPO 3 É O QUE **TODO MUNDO ESQUECE**.



O MESMO CONTRATO, QUATRO PACOTES

Pacote	Pedir	Ler	Parar
expo-location	requestForegroundPermissionsAsync()	getCurrentPositionAsync() watchPositionAsync()	subscription.remove()
expo-sensors	—	Accelerometer.addListener() Gyroscope.addListener()	subscription.remove()
expo-camera	useCameraPermissions()	takePictureAsync()	desmontar o CameraView
expo-image	—	—	—



POR QUE ACELERÔMETRO E GIROSCÓPIO NÃO PEDEM PERMISSÃO?

PERGUNTE À TURMA ANTES DE RESPONDER.



PERMISSÃO É UM ESTADO, NÃO UM EVENTO

O usuário pode revogar nas Configurações **com o app rodando**.

Campo	Para quê
status	'granted' / 'denied' / 'undetermined'
granted	o mesmo, como booleano
canAskAgain	se false , o diálogo não aparece mais
expires	normalmente 'never'

CONSULTE TODA VEZ QUE FOR USAR.



canAskAgain: false

O sistema **não vai mostrar o diálogo de novo** — por mais vezes que você chame `request...`. Seu botão "Permitir" virou um botão que não faz nada.

A única saída:

- 1 **Explicar** por que o app precisa daquilo
- 2 **Levar às Configurações** do sistema

INSISTIR COM O MESMO BOTÃO É O QUE A MAIORIA DOS APPS FAZ. E ESTÁ ERRADO.

AS DUAS FORMAS DE PERGUNTAR

Quando importa só na hora da ação:

```
const { status } =  
  await Location  
    .requestForegroundPermissionsAsync();
```

Quando a tela inteira depende dela:

```
const [permissao, pedirPermissao] =  
  useCameraPermissions();
```

O SEGUNDO DEVOLVE O MESMO FORMATO DO `useState`. NÃO É CONCEITO NOVO.



O COPO E A TORNEIRA

`getCurrentPositionAsync` ENCHE UM COPO. `watchPositionAsync` ABRE A TORNEIRA.

A PERGUNTA QUE DECIDE: UM VALOR, OU UM FLUXO?

Situação	O quê
"Onde eu estou agora ?"	copo – <code>getCurrentPositionAsync</code>
"O trajeto enquanto eu corro"	torneira – <code>watchPositionAsync</code>
"O usuário chacoalhou?"	torneira – <code>addListener</code> (não há copo)
"Tirar uma foto"	copo – <code>takePictureAsync</code>

 **TODA TORNEIRA ABERTA TEM UMA TORNEIRA FECHADA NO MESMO ARQUIVO.**



NOTA HONESTA: O QUE AINDA NÃO TEMOS

Num app de verdade, a torneira abre quando a tela aparece e fecha quando ela sai. A ferramenta do React que faz isso é assunto da **Aula 6**.

Hoje: você liga e desliga **no botão**, e guarda a assinatura num `useState`.

Se você sair da tela com o sensor ligado, ele **continua ligado**.

0 app.json – ESCREVA AGORA, NÃO EM NOVEMBRO

```
["expo-location", {  
  "locationWhenInUsePermission":  
    "Precisamos da sua localização para registrar onde o treino aconteceu."  
}]
```

No **Expo Go** funciona **mesmo sem** isso – ele herda as permissões do próprio Expo Go. Na **build de verdade** (Aula 13), **não funciona**.

E ESCREVA UM TEXTO HONESTO. O USUÁRIO LÊ ANTES DE DECIDIR.

PARTE 2

GPS COM expo-location



`npx expo install`, **NÃO** `npm install`

```
npx expo install expo-location expo-sensors expo-camera expo-image
```

O `expo install` escolhe a versão **compatível com o seu SDK**. O `npm install` pega a mais nova — que pode não ser essa.

"VERSÃO MAIS NOVA ≠ VERSÃO QUE VOCÊ USA" — AULA 1, AGORA NO SEU TERMINAL.

O COPO

```
import * as Location from 'expo-location';

const { status, canAskAgain } =
  await Location.requestForegroundPermissionsAsync();

const posicao = await Location.getCurrentPositionAsync({
  accuracy: Location.Accuracy.Balanced,
});
```

 DENTRO DE PRÉDIO, COM O GPS FRIO: VÁRIOS SEGUNDOS.



Accuracy : PRECISÃO É UMA COMPRA

Nível	Erro	Custo
Lowest	~3 km	mínimo
Low	~1 km	baixo
Balanced	~100 m	médio – default
High	~10 m	alto
Highest	o melhor disponível	muito alto
BestForNavigation	o melhor, para navegação	o mais caro

VOCÊ PAGA EM TEMPO DE ESPERA E EM BATERIA.



ESCOLHA EM VOZ ALTA

O app faz	O que pedir
Previsão do tempo da cidade	Low
Em que academia o treino foi	Balanced ou High
Trajetos de uma corrida	High · BestForNavigation

 O ERRO: Highest "POR GARANTIA". GARANTIA DE GASTAR BATERIA.



accuracy NÃO É NOTA DE QUALIDADE

```
coords: {  
  latitude, longitude,  
  accuracy,          // RAI0 de incerteza, em METROS  
  altitude, altitudeAccuracy,  
  heading,           // graus – só faz sentido em movimento  
  speed              // m/s – idem  
}
```

accuracy: 65 = "EM ALGUM LUGAR NUM CÍRCULO DE 65 METROS".

O PADRÃO DE UX QUE VALE A AULA INTEIRA

```
const ultima = await Location.getLastKnownPositionAsync({ maxAge: 60000 });
if (ultima) setPosicao(ultima);           // algo na tela AGORA

const atual = await Location.getCurrentPositionAsync({ /* ... */ });
setPosicao(atual);                        // o dado bom, quando chegar
```

50 MS DE RESPOSTA EM VEZ DE 6 SEGUNDOS DE SPINNER.

A TORNEIRA

```
const assinatura = await Location.watchPositionAsync(  
  { accuracy: Location.Accuracy.High, TimeInterval: 5000, distanceInterval: 10 },  
  (posicao) => setPosicao(posicao),  
);  
  
assinatura.remove(); // ← não é opcional
```

`TimeInterval` e `distanceInterval` são **filtros**.

`TimeInterval` é **Android-only**. No iPhone ele é ignorado — compila, não avisa, não faz nada.
`distanceInterval` vale nos dois.



PERMISSÃO CONCEDIDA ≠ GPS LIGADO

```
const ligado = await Location.hasServicesEnabledAsync();
```

Situação	O usuário resolve em
Permissão negada	Configurações do app
Serviço desligado	Configurações do sistema

UMA MENSAGEM GENÉRICA PARA OS DOIS = DESINSTALAÇÃO.

COORDENADA → ENDEREÇO

```
const enderecos = await Location.reverseGeocodeAsync({ latitude, longitude });  
// street, district, city, region, postalCode, country
```

"Boa Viagem, Recife" em vez de `-8.1234, -34.9012`.

A documentação avisa: geocoding é **caro**. Uma chamada por registro — **nunca** dentro do `watchPositionAsync`.



FORA DO ESCOPO – E A TURMA VAI PERGUNTAR

Assunto	Por quê
Mostrar num mapa	biblioteca de terceiro, não está no cronograma
Localização em segundo plano	exige development build
Geofencing	idem, e depende do <code>expo-task-manager</code>

HOJE, A COORDENADA É TEXTO.



CHECKPOINT 1

60 SEGUNDOS EM DUPLAS, E A GENTE RESPONDE EM VOZ ALTA.

CHECKPOINT 1 – PERMISSÃO E GPS

- 1 Quais são os **três tempos**? Qual não existe no `getCurrentPositionAsync` ?
- 2 `canAskAgain: false` . O que o seu botão "Permitir" deve fazer?
- 3 `accuracy: 65` . O que é esse 65? Bom ou ruim?
- 4 Previsão do tempo da cidade: qual `Accuracy` , e por que **não** `Highest` ?
- 5 "Permissão negada" × "serviço desligado": a mensagem pode ser a mesma?



PARTE 3

ACELERÔMETRO E GIROSCÓPIO



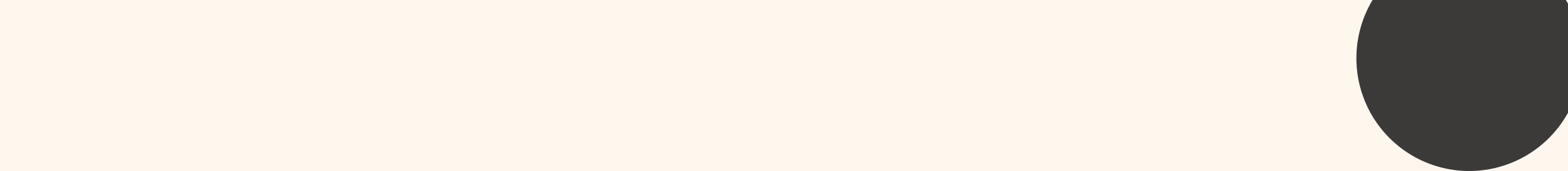


O PACOTE NÃO SE CHAMA expo-gyroscope

O cronograma da disciplina diz "Expo Gyroscope". **Esse pacote não existe.**

```
import { Accelerometer, Gyroscope } from 'expo-sensors';
```

QUANDO UM NOME NÃO RESOLVE, PROCURE O PACOTE GUARDA-CHUVA.



O QUE CADA UM MEDE

	Acelerômetro	Giroscópio
Mede	translação + gravidade	rotação
Unidade	g (1 g = 9,81 m/s ²)	rad/s
Parado na mesa	não dá zero	dá zero
Responde a	empurrão, chacoalhada, inclinação	"estou girando, e quão rápido"





O PASSAGEIRO DO ÔNIBUS

O **SOLAVANCO** DA FREADA É O ACELERÔMETRO. O **RODOPIO** DA CURVA É O GIROSCÓPIO.



O ACELERÔMETRO PARADO NÃO MARCA ZERO

Telefone parado na mesa:

- **Giroscópio:** ≈ 0 nos três eixos
- **Acelerômetro:** magnitude $\approx 1\text{ g}$

Não é bug. É a **gravidade**, que não desliga. E é por isso que o acelerômetro serve para detectar **inclinação**.

A API – A MESMA PARA OS DOIS

```
const disponivel = await Gyroscope.isAvailableAsync(); // simulador não tem

Gyroscope.setUpdateInterval(100);

const assinatura = Gyroscope.addListener(({ x, y, z, timestamp }) => {
  setDados({ x, y, z });
});

assinatura.remove();
```

● **removeAllListeners()** ESTÁ DEPRECADO. QUASE TODO TUTORIAL USA.



setUpdateInterval É UM PEDIDO

```
Accelerometer.setUpdateInterval(16);  
Accelerometer.addListener(setDados); // ✗ 60 setState por segundo
```

60 setState /s = 60 renderizações/s. Numa tela com **FlatList**, derruba o app.

⚠️ Android 12+ limita a 200 Hz por padrão — proteção de privacidade.

AS DUAS CORREÇÕES

1. **Aumente o intervalo.** O olho não lê 60 números por segundo.

```
Accelerometer.setUpdateInterval(100); // ✓ 10 Hz
```

2. Só chame `setState` quando o resultado mudar.

```
setChacoalhou((anterior) => (anterior === agora ? anterior : agora));
```



A FERRAMENTA "CERTA" PARA ISSO USA RECURSOS QUE AINDA NÃO VIMOS.



CHACOALHADA: USE A MAGNITUDE

```
const magnitude = Math.sqrt(x * x + y * y + z * z); // parado ≈ 1 g
const chacoalhou = magnitude > 1.8;                // limiar EMPÍRICO
```

Olhando só o `x`, uma chacoalhada de cima para baixo **passa batido**.

A MAGNITUDE É INDEPENDENTE DA ORIENTAÇÃO DO APARELHO.

SIMULADOR E EMULADOR

Ambiente	Acelerômetro / Giroscópio
Aparelho físico	✅ o único jeito de verdade
Android Emulator	⚠️ sensores virtuais nos extended controls
iOS Simulator	❌ não tem – é para isso que serve <code>isAvailableAsync()</code>

ESTE É O BLOCO QUE MENOS TOLERA SIMULADOR.





PARTE 4

CÂMERA COM expo-camera

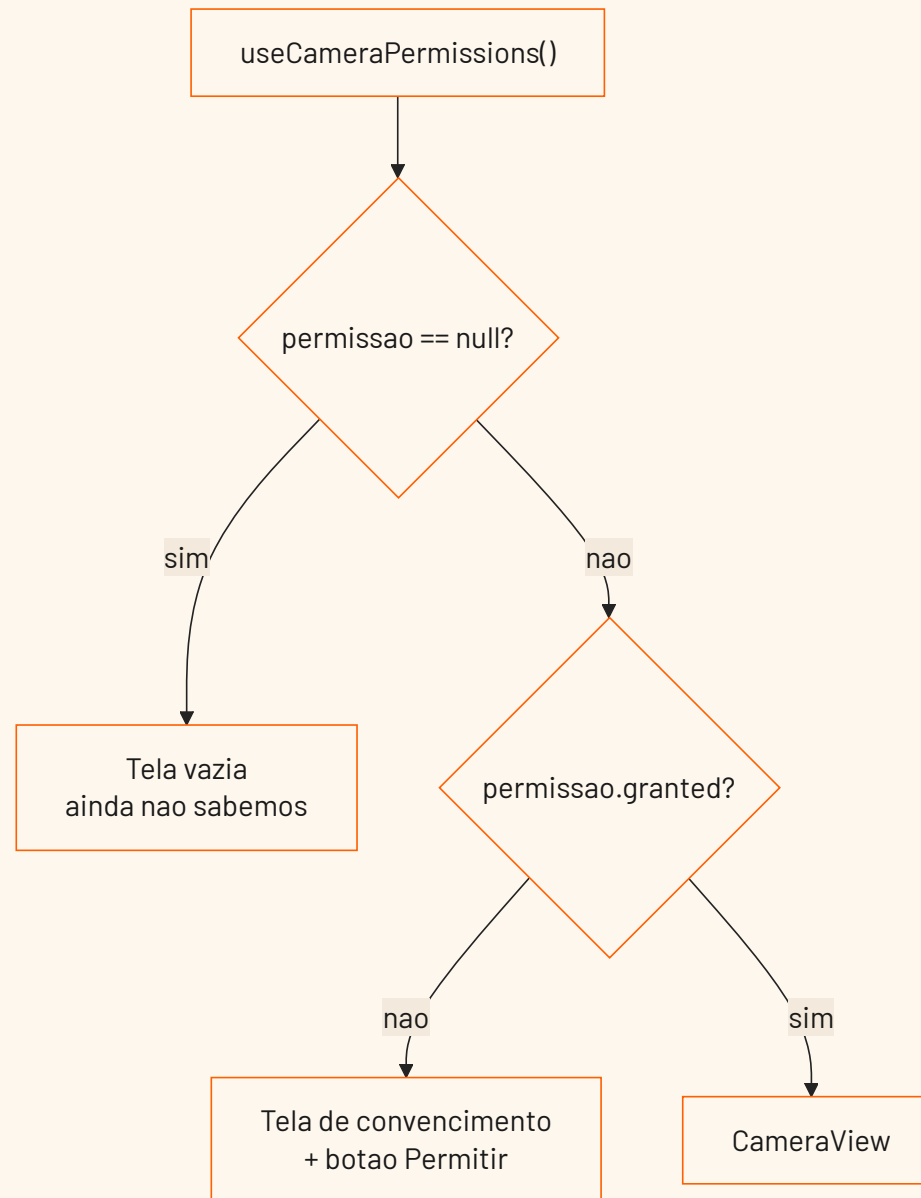


O TUTORIAL QUE VOCÊ VAI ACHAR NÃO COMPILA

O tutorial de 2023	O que compila hoje
<code>import { Camera }</code>	<code>import { CameraView, useCameraPermissions }</code>
<code><Camera type={Camera.Constants.Type.back} /></code>	<code><CameraView facing="back" /></code>
<code>CameraType.back</code> (enum)	a string <code>'back'</code>
<code>Camera.requestCameraPermissionsAsync()</code>	<code>useCameraPermissions()</code>
<code>expo-camera/legacy</code>	removido no SDK 52
<code>onBarcodeScanned</code>	<code>onBarcodeScanned</code>

QUANDO UM EXEMPLO NÃO COMPILA: ABRA A DOCUMENTAÇÃO DA SUA VERSÃO.

O GATE DE PERMISSÃO TEM TRÊS ESTADOS



SÃO DUAS PERGUNTAS, NÃO UMA.

OS TRÊS CAMINHOS DE `return`

```
const [permissao, pedirPermissao] = useCameraPermissions();

if (!permissao) return <View />;           // ainda não sabemos

if (!permissao.granted) {                  // sabemos, e não temos
  return (
    <View style={estilos.centro}>
      <Text>Precisamos da câmera para registrar sua foto.</Text>
      <Pressable onPress={pedirPermissao}><Text>Permitir</Text></Pressable>
    </View>
  );
}

return <CameraView style={estilos.camera} facing={lado} />;
```

O ÚNICO NOME NOVO AQUI É O `useCameraPermissions` .



`null` NÃO É "NEGADO"

`permissao` começa `null` = "a resposta ainda não chegou".

Se você juntar os dois primeiros casos, a tela de "precisamos da câmera" **pisca para todo mundo** — inclusive para quem autorizou há semanas.

A TELA DO ESTADO 2 NÃO É UMA TELA DE ERRO. É O SEU ARGUMENTO DE VENDA.



CameraView SEM ALTURA = TELA PRETA

```
const estilos = StyleSheet.create({  
  camera: { flex: 1 }, // ← sem isso, você não vê nada  
});
```

ANTES DE DEBUGAR PERMISSÃO, CONFIRA O ESTILO.

AS PROPS QUE SÃO ESTADO

Prop	Valores	Default
facing	'back' / 'front'	'back'
flash	'off' / 'on' / 'auto' / 'screen'	'off'
enableTorch	booleano	false
zoom	0 a 1	0
mode	'picture' / 'video'	'picture'

● zoom VAI DE 0 A 1, NÃO "2X". E flash ≠ enableTorch .



TIRAR A FOTO PRECISA DE UMA REFERÊNCIA

A forma canônica é `useRef`. Ela é **Aula 6**.

```
const [camera, setCamera] = useState<CameraView | null>(null);  
  
<CameraView ref={setCamera} style={estilos.camera} facing={lado} />
```

- 1 A prop `ref` **aceita uma função**
- 2 O React **chama** essa função com o componente montado
- 3 `setCamera` **é** uma função

NÃO É HOOK NOVO. É O `useState` QUE VOCÊS JÁ USAM.

takePictureAsync

```
async function tirarFoto() {  
  if (!camera) return; // ainda é null no 1º render  
  const foto = await camera.takePictureAsync({ quality: 0.7 });  
  if (foto) setFotoUri(foto.uri);  
}
```

```
{ uri, width, height, base64?, exif?, format }
```

NÃO PEÇA `base64` SEM PRECISAR. PARA EXIBIR, O `uri` BASTA.



A FOTO NÃO FOI PARA A GALERIA

O `uri` aponta para o **cache do aplicativo**: `file:///.../Caches/...`

Se o sistema limpar o cache, **ela some**. Salvar de verdade é outro pacote, e não está no nosso programa.

VOCÊS PRECISAM SABER DISSO PARA NÃO PROMETEREM AO USUÁRIO ALGO QUE O APP NÃO FAZ.

PARTE 5

`expo-image`

"A AULA 3 JÁ ENSINOU `Image` . POR QUE OUTRO?"

O expo-image traz	O problema que resolve
Cache em memória e disco	a mesma imagem baixada de novo a cada rolagem
<code>placeholder</code>	o retângulo cinza piscando
<code>transition</code>	a imagem aparecendo com corte seco
<code>contentFit</code>	recorte com a semântica do CSS
<code>recyclingKey</code>	a foto do item anterior aparecendo

NÃO É MODA. É A LISTA DE PROBLEMAS QUE APARECEM QUANDO O APP CRESCE.



FECHANDO O CICLO DA AULA

```
import { Image } from 'expo-image';

<Image
  source={{ uri: fotoUri }}
  style={estilos.previa}
  contentFit="cover"
  transition={300}
/>
```

SENSOR → DADO → TELA. O `uri` DO `takePictureAsync` ENTRA AQUI.



Image E Image TÊM O MESMO NOME

- Importar os dois no mesmo arquivo → **erro de compilação**
- Importar o **errado** → **bug silencioso**: `contentFit` não faz nada

"O `contentFit` não funciona" → confira o **import**. É a causa nº 1.

 resizeMode ESTÁ DEPRECADO NO expo-image

contentFit	Comportamento
cover	preenche, cortando – default
contain	cabe inteira, com sobra
fill	estica, distorcendo
none · scale-down	sem ajuste · o menor entre os dois

OS VALORES AGORA SÃO OS DO CSS object-fit – IGUAIS NAS TRÊS PLATAFORMAS.

placeholder E transition

```
<Image  
  source={{ uri: fotoUri }}  
  placeholder={{ blurhash }}  
  placeholderContentFit="cover"  
  transition={300}  
>
```

Blurhash: uma **string curta** que representa como a imagem parece.

O PROBLEMA RESOLVIDO É DE LAYOUT: SEM PLACEHOLDER, A LISTA PULA.



cachePolicy – A COZINHA

Valor	Onde
'none'	não guarda
'disk'	default
'memory'	some ao fechar o app
'memory-disk'	os dois

MEMÓRIA = A BANCADA. DISCO = O ARMÁRIO. REDE = O SUPERMERCADO.



QUANDO NÃO TROCAR

Uma logo local com `require()` **não precisa de nada disso**. Já está no bundle.

Troque quando houver: imagem **remota**, **lista**, ou **espera perceptível**. Fora disso, o `Image` da Aula 3 continua correto.

SABER QUANDO NÃO USAR UMA FERRAMENTA É PARTE DE SABER USÁ-LA.



👋 CHECKPOINT 2

CHECKPOINT 2 – SENSORES, CÂMERA E IMAGEM

- 1 Telefone parado na mesa: o que marca o giroscópio? E o acelerômetro? Por quê?
- 2 Detectar chacoalhada: qual sensor, e por que a **magnitude**?
- 3 Por que o gate da câmera tem **três** `return` s? O que quebra se juntar dois?
- 4 Como tiramos a foto **sem** `useRef` ? Qual a desvantagem?
- 5 `contentFit` não faz efeito. Qual é a **primeira** coisa que você confere?

HANDS-ON – 25 MIN

`exercises.md` (Hábito) ou `practice.md` (Pet)

#	Exercício	Tempo
1	Associação: qual pacote resolve o sintoma	4 min
2	Caça ao erro: a tela de câmera de 2023	5 min
3	Complete: o botão "Onde eu estou"	6 min
4	Complete: o detector de chacoalhada	6 min
5	Complete: câmera e prévia	4 min



ATIVIDADE APLICADA – "O REGISTRO COM CONTEXTO"

A mesma lista da Aula 4, agora com dados que **ninguém precisou digitar**.

O que se avalia:

- 1 Os **três tempos** aparecem no código: pede, lê, **para**
- 2 Os estados de falha estão na tela, e são **diferentes** entre si
- 3 A `Accuracy` está **justificada** no README
- 4 Nenhuma torneira aberta sem `remove()`
- 5 `contentFit` explícito, import conferido



VOCÊS LIGARAM O SENSOR **NO BRAÇO, COM UM BOTÃO.**

E EU ADMITI DUAS VEZES QUE ELE CONTINUA LIGADO QUANDO VOCÊS SAEM DA TELA.

ATALHOS DESTA APRESENTAÇÃO

Tecla	Ação
→ / espaço · ←	avancar · voltar
S	modo apresentador (notas + cronômetro)
F	tela cheia
0 ou Esc	visão geral dos slides
B ou .	tela preta
?	todos os atalhos